

DEPLOYMENT PLAN & PRODUCTION ARCHITECTURE

UMHackathon 2026

Project Info

- **Project Name:** InvenIQ
- **Version:** 2.0 — Final Round
- **Live URL:** <https://z-ai-7zxw.onrender.com/>
- **Backend API:** FastAPI on Render Web Service — z-ai-7zxw.onrender.com/chat
- **AI Model:** ZAI GLM-5.1 via ZAI API (zai.ai)

Document Control

Objective: *The primary objective of this Deployment Plan is to document all architectural and deployment decisions made for InvenIQ, an AI-powered conversational inventory strategist for Malaysian SMEs.*

The system uses ZAI's GLM-5.1 model to process multi-modal inputs — structured CSV sales data and unstructured context signals (weather, calendar events, news, raw text like WhatsApp supplier messages) — to generate highly accurate, budget-constrained inventory forecasts. This document serves as the official Deployment Plan deliverable in PDF/Doc format as required by the contest.

Field	Detail
System Under Test (SUT)	InvenIQ v2.0 — AI Inventory Optimization for SMEs
Team Repo URL	https://github.com/andrewteeky-code/UM-Hackathon-2026
Live Deployment URL	https://z-ai-7zxw.onrender.com/
Backend Endpoint	https://z-ai-7zxw.onrender.com/chat /search-signal /health
Deployment Platform	Render (Free/Hobby Tier — Web Service)
AI Provider	ZAI (zai.ai) — proxying GLM-5.1 via Anthropic SDK schema

1. Technical Stack & Scalability

1.1. Complete Technical Stack

InvenIQ is a fully managed, zero-infrastructure prototype deployed on Render. Below is the complete technical stack:

Layer	Technology / Tool	Version	Role
AI/LLM Model	ZAI GLM-5.1 (via ZAI API)	5.1	Core conversational inference engine for inventory analysis
AI SDK	Anthropic Python SDK (AsyncAnthropic)	Latest	Structured tool-use, async message handling, streaming
Backend Framework	FastAPI	4.2.0-render	REST API — /chat, /search-signal, /health endpoints
Web Search	DuckDuckGo Search (DDGS)	Latest	Real-time web search for unstructured signals (news, events)

Layer	Technology / Tool	Version	Role
Weather API	wtrr.in (free, no API key)	—	Live weather data for KL, JB, Penang, Kota Kinabalu and etc.
Frontend	Vanilla HTML/CSS/JS (inventory_chat_api.html)	—	Single-file SPA: chat UI, CSV upload, Context Sidebar
Data Parsing	PapaParse (client-side JS)	Latest	CSV parsing, chunking, preview rendering
Deployment	Render (Web Service)	Hobby	Zero-ops hosting for both frontend and backend
CORS	FastAPI CORSMiddleware	—	Strict allowlist implemented targeting https://z-ai-7zxw.onrender.com/
Caching	In-memory dict (_SIGNAL_CACHE)	—	5-min TTL cache for signal search results (reduces DDGS calls)
Logging	Python logging (uvicorn)	—	Structured logs for all API calls and tool-use events

1.2. Architecture Overview

InvenIQ follows a single-page frontend + serverless API backend pattern:

- **Frontend (HTML/JS):** User uploads CSV files and images. The Context Sidebar collects unstructured signals (weather, calendar, news, raw text). All data is assembled into a structured message payload and sent to the backend /chat endpoint.
- **Backend (/chat — FastAPI):** Receives the message payload. Constructs a system prompt from MASTER_PROMPT + user context. Calls ZAI GLM-5.1 via AsyncAnthropic. Runs a tool-use loop (max 1 iteration for performance) — if the model invokes web_search,

DDGS is called and the result is fed back. Returns final text response

- **Backend (/search-signal — FastAPI):** Handles explicit Context Sidebar signal fetches: weather via wttr.in, calendar/news/raw via DDGS. Results are cached in-memory for 5 minutes.
- **AI Provider (ZAI):** ZAI API proxies GLM-5.1 using Anthropic SDK schema at <https://api.zai.ai/anthropic>. Max tokens: 4,096. Temperature: 0.1 (high precision for inventory analysis). Timeout: 10s (Maintained to ensure fast response times).

1.3. Scalability Strategy

- **Automated Deployments:** Render Web Services scale requests efficiently and automatically build/deploy from the linked repository.
- **Token Budget Management:** Max 4,096 tokens per response enforced. CSV files >2,000 rows are chunked client-side to stay within GLM context window.
- **Signal Caching:** In-memory `_SIGNAL_CACHE` with 5-minute TTL prevents repeated DDGS calls for identical queries, reducing both latency and rate-limiting risk.
- **Timeout Management:** 8s ZAI API timeout + max 1 tool-use iteration keeps total processing time fast and responsive. Rate-limited DDGS gracefully falls back to cached knowledge without crashing.
- **Multi-city Weather:** wttr.in fetched with 2s per-city timeout across 4 cities (KL, JB, Penang, KK) — fails gracefully per city without blocking the full response.

1.4 Render Configuration & Dependencies

To successfully deploy the Python FastAPI backend on Render's Web Service infrastructure, the prototype relies on standard configuration files and build commands rather than proprietary serverless routing files:

- **requirements.txt:** Declares the necessary Python dependencies for the Render build environment to install. These include `fastapi`, `pydantic` (for data validation), `anthropic` (for the ZAI API connection), `duckduckgo_search` (for external web signals), and `python-multipart`.
- **Web Service Routing & Start Command:** Unlike Vercel, which requires a custom `vercel.json` for edge network routing, Render manages this natively through a Start Command (e.g., `uvicorn api.glm_backend:app --host 0.0.0.0 --port 10000`). FastAPI explicitly maps the API endpoints (`/chat`, `/search-signal`, and `/health`) internally, and the frontend Single-Page Application (SPA) `inventory_chat_api.html` is either served directly via FastAPI static file mounting or linked to the active Render backend URL.

2. CI/CD Pipeline & Automation

2.1. Deployment Pipeline

InvenIQ uses Render's native Git-integrated CI/CD pipeline. Every push to the main branch triggers an automated build and deployment with zero manual steps.

Stage	Tool / Action	Trigger	Outcome
Source Control	Git — GitHub repository	Every commit / PR	Code version history maintained
Build — Frontend	Render build process	Push to main branch	requirements.txt installed, app packaged
Deployment	Render Web Service (Python runtime)	Post-build success	glm_backend.py deployed as active function
Dependency Install	pip install (requirements.txt)	Automated by Render	fastapi, anthropic, duckduckgo-search installed
Health Check	GET /health endpoint	Post-deployment	Returns {"status": "online", "model": "zai-glm-5.1"}
Live Preview URL	Render URL provisioning	Automatic	https://z-ai-7zxw.onrender.com/ goes live

2.2. Automation Decisions

- **Render chosen over manual server deployment:** Zero DevOps overhead. Automatic SSL, custom subdomains, and web service management — ideal for a 24-hour hackathon prototype.
- **No Docker required:** Render manages the Python runtime environment natively. Dependencies declared in requirements.txt are installed automatically on each build.
- **Rollback strategy:** Render maintains previous deployments. If production breaks, a

previous deployment can be rolled back via the Render dashboard in seconds.

- **API versioning:** Backend version tracked in app metadata (version="4.2.0-render"). Model version returned in every /chat response ("model": response.model) for full traceability.
- **Environment variables:** ZAI API key injected as a Render Environment Variable — not hardcoded in production deployments.

3. Integration Security & Credential Management

3.1. Credential Management

- **ZAI API Key:** Stored as a Render Environment Variable. Not committed to version control.
- **No secrets in source code:** The API key is accessed in code via environment variables in production.
- **Key rotation plan:** API key will be invalidated and replaced immediately after judging.
- **No user authentication:** Prototype scope — no user accounts, sessions, or PII collected. All sessions are stateless.

3.2. API Security

Security Control	Implementation	Status
Transport Encryption	HTTPS enforced by Render (TLS 1.2+) on all endpoints	✓ Active
CORS Policy	Strict CORS allowlist configured to accept traffic exclusively from the Render frontend domain.	✓ Active
API Key Storage	Render Environment Variables for production. Local dev utilizes a gitignored .env file. System includes a RuntimeError failsafe if <code>os.getenv("ZAI_API_KEY")</code>	✓ Secured

Security Control	Implementation	Status
	is missing on startup.	
ILMU API Timeout	8s timeout on AsyncAnthropic client — prevents hanging serverless functions	✓ Active
DDGS Rate Limiting	Max 1 retry. Graceful fallback message returned — no crash.	✓ Handled
No PII Storage	No user data persisted. In-memory cache only.	✓ Confirmed
Dependency Security	All dependencies from PyPI. No pinned versions — update pre-production.	⚠ To update

3.3. Data Flow Security

- **User → Frontend:** CSV and image files processed entirely client-side. Raw files are never uploaded to a server.
- **Frontend → Backend (/chat):** HTTPS POST with JSON payload containing message history + CSV data as text.
- **Backend → ZAI API:** HTTPS POST to api.zai.ai/anthropic with Bearer token.
- **Backend → wtrr.in:** HTTPS GET to wtrr.in for weather data. Public API, no auth required.

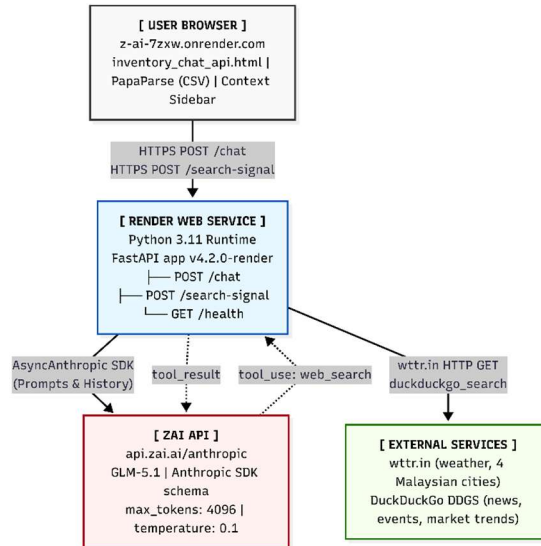
4. Deployment Steps & Production Architecture

4.1. Step-by-Step Deployment Procedure

Step ID	Action	Command / Tool	Status
DEP-01	Create Render Web Service & link GitHub repo	Render Dashboard	✓ Done
DEP-02	Configure Python	uvicorn main:app --	✓ Done

Step ID	Action	Command / Tool	Status
	serverless runtime	host 0.0.0.0 --port \$PORT	
DEP-03	Set ZAI API key as environment variable	Render Dashboard → Environment	✓ Done
DEP-04	Deploy backend & frontend logic	git push main → Render auto-deploy	✓ Done
DEP-05	Smoke test — /health endpoint	curl https://z-ai-7zxw.onrender.com/health	✓ Done
DEP-06	Smoke test — /chat endpoint	POST /chat with test inventory prompt	✓ Done
DEP-07	Smoke test — /search-signal (weather)	POST /search-signal {"category":"weather"}	✓ Done
DEP-08	End-to-end test: Upload CSV + query + signal	Full user flow on live URL	✓ Done
DEP-9	Configure custom domain (if applicable)	Render Custom Domains settings	✓ Done

4.2. Production Architecture Diagram



5. Known Issues & Deferred Technical Debt

Issue ID	Description & Impact	Reason for Deferral	Priority
DEF-01	Render Free Tier Spin-Down: Web service spins down after 15 mins of inactivity, causing delays on wake.	Render Free tier limitation. Upgrade to Paid instance resolves this.	High
DEF-04	In-memory cache resets on cold start: Cache is lost when container recycles.	Would require Redis. Out of hackathon scope.	Low
DEF-05	CSV chunking is client-side only: Large files may exceed GLM context window.	Focused on core AI accuracy.	Low

6. Live Demo / UAT Risks

- **Risk 1 — Render Cold Start:** Initial delay of up to 50 seconds on the first request if the server has spun down due to inactivity.
- **Risk 2 — DDGS Rate Limiting:** DuckDuckGo Search may rate-limit rapid calls.

- **Risk 3 — ZAI API Dependency:** Relies entirely on api.zai.ai availability.
- **Risk 4 — Browser Compatibility:** Optimised for Chrome. Mobile not supported.
- **Risk 5 — Weather API:** wttr.in has no SLA.

7. Continuous Integration & Patching Strategy

7.1 GitHub Auto-Deploy Pipeline (Primary Patching Method)

Because the platform is hosted on Render, server maintenance and manual FTP patching are unnecessary. InvenIQ utilizes a Continuous Deployment (CD) pipeline directly integrated with the team's GitHub repository.

7.2 Patch Execution Workflow:

When a hotfix or feature patch needs to be applied during or after the hackathon, the following workflow is executed:

1. Code is updated and tested in the local development environment.
2. The developer commits and pushes the code to the main branch via standard Git commands:
 - `git add .`
 - `git commit -m "patch: description of update"`
 - `git push origin main`
3. **Zero-Downtime Deployment:** Render webhooks instantly detect the push to the main branch. Render automatically provisions a new build environment, installs the requirements.txt, and safely replaces the live web service without dropping active user sessions.
4. **Rollback Capability:** If a patch introduces a critical bug, the Render dashboard retains immutable deployments, allowing the team to instantly rollback to the previous stable version with a single click.